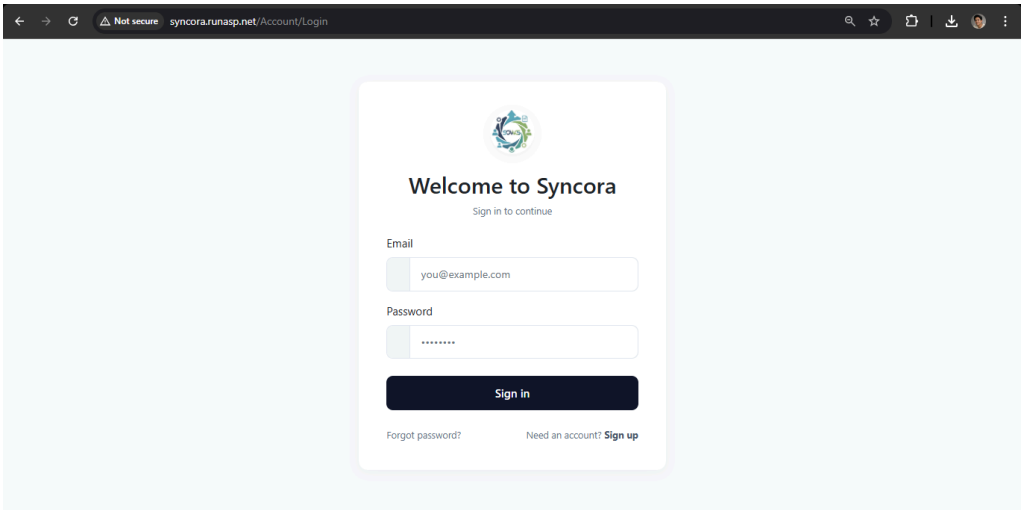
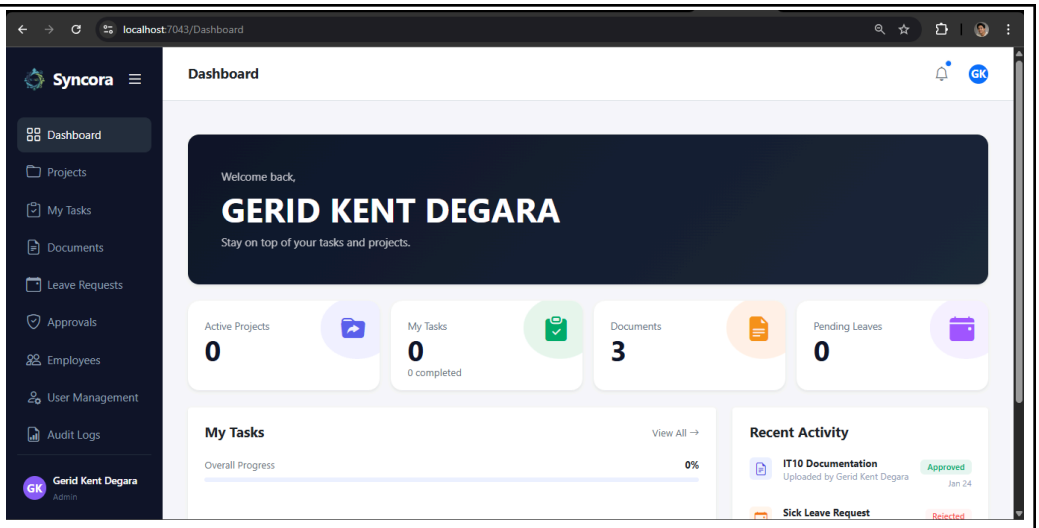
NAME	**GERID KENT P. DEGARA**
PROJECT TITLE	**Syncora Office Workflow & Collaboration System**
SUBJECT: **CODE:** **TIME:**	**IT15/L Integrative Programming and Technologies** **8460** **330A-530E**
TOPIC (Type of Business Process)	**#37 Collaborative and Workflow Transactions**
Products/Services	**Workflow Automation Service, Project Management Service, Document Management & Collaboration Service, HR Self-Service Platform**
Deployed Link / Website link	**http://syncora.runasp.net/**
Role Based-Access	**Super Admin Acct. (if done)** **Username: yuzkiega@gmail.com** **Password: Hahaha123@** **Admin Acct.** **Username: admin@gmail.com** **Password: Hahaha123@** **Client Acct.** **Username: client@gmail.com** **Password: Hahaha123@**
3rd Deliverables-Deployed Transactions (CRUD)	
Log in-Page (Screenshot)	 . This image shows a login page for a website called Syncora. Users can enter their registered email and password into the boxes to access their accounts. There are also options to sign up for a new account or reset a forgotten password.

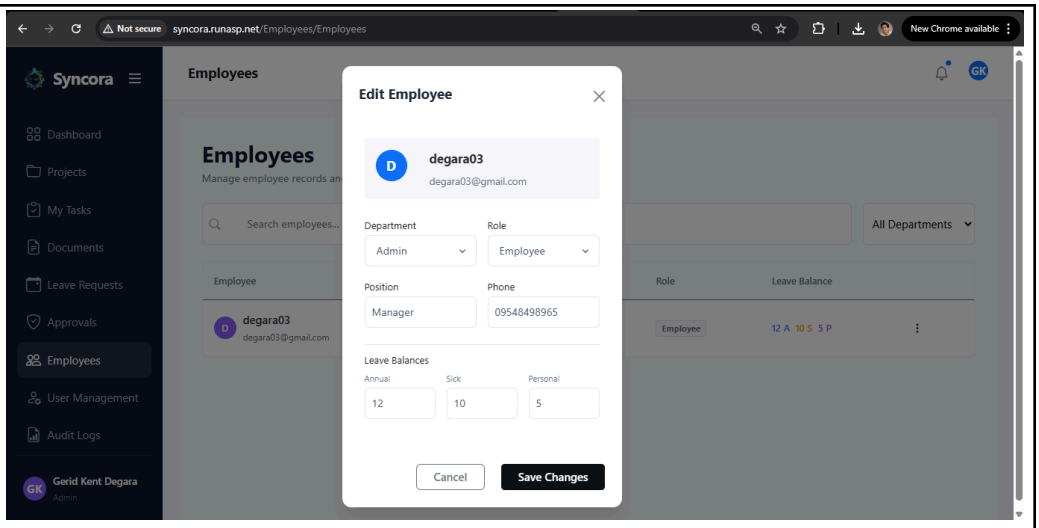
Log in-Page - Source Code (Screenshot)	

```
using IT15_SOWCS.Models;
using IT15_SOWCS.ViewModels;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
namespace IT15_SOWCS.Controllers
{
    public class AccountController : Controller
    {
        private readonly SignInManager<Users> signInManager;
        private readonly UserManager<Users> userManager;

        public AccountController(SignInManager<Users> signInManager, UserManager<Users> userManager)
        {
            this.signInManager = signInManager;
            this.userManager = userManager;
        }

        public IActionResult Login()
        {
            return View();
        }
        [HttpPost]
        public async Task<IActionResult> Login(LoginViewModel model)
        {
            if (ModelState.IsValid)
            {
                var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
model.RememberMe, false);
                if (result.Succeeded)
                {
                    return RedirectToAction("Index", "Dashboard");
                }
                else
                {
                    ModelState.AddModelError("", "Wrong email or password.");
                    return View(model);
                }
            }
            return View(model);
        }
        public IActionResult Register()
        {
            return View();
        }
        [HttpPost]
        public async Task<IActionResult> Register(RegisterViewModel model)
        {
            if (ModelState.IsValid)
            {
                Users user = new Users
                {
                    UserName = model.Email,
                    Email = model.Email,
                    FullName = model.FullName
                };

                var result = await userManager.CreateAsync(user, model.Password);

                if (result.Succeeded)
                {
                    return RedirectToAction("Login", "Account");
```

Dashboard (Client/End-User) (Screenshot)	 This image displays the main dashboard of the Syncora platform for client/end-user, where users can oversee their current workload and company updates. It features summary cards for active projects, tasks, and documents, alongside a sidebar for navigating to specific sections." The interface also includes a Recent Activity feed to track the latest status changes on submitted files and requests.

Dashboard - Source Code **(Client/End-User) (Screenshot)**	

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using IT15_SOWCS.Data;
using IT15_SOWCS.Models;
using System.Security.Claims;
using System.Threading.Tasks;

namespace IT15_SOWCS.Controllers
{
    [Authorize]
    public class DashboardController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly UserManager<Users> _userManager;

        public DashboardController(
            ApplicationDbContext context,
            UserManager<Users> userManager)
        {
            _context = context;
            _userManager = userManager;
        }

        public async Task<IActionResult> Index()
        {
            var currentUser = await _userManager.GetUserAsync(User);
            if (currentUser == null)
            {
                return RedirectToAction("Login", "Account");
            }

            var employee = await _context.Employees
                .FirstOrDefaultAsync(e => e.UserId == currentUser.Id);

            if (employee == null)
            {
                ViewBag.FullName = currentUser.FullName ?? currentUser.UserName ?? "User";
                ViewBag.ShowNoEmployeeProfile = true;
                return View();
            }

            var activeProjectsCount = await _context.Projects
                .CountAsync(p => p.IsActive &&
                    (p.ManagerEmail == employee.User.Email ||
                     _context.Tasks.Any(t => t.ProjectId == p.ProjectId &&
                                            t.AssignedTo == employee.EmployeeId)));

            var myTasksCount = await _context.Tasks
                .CountAsync(t => t.AssignedTo == employee.EmployeeId);

            var myCompletedTasksCount = await _context.Tasks
                .CountAsync(t => t.AssignedTo == employee.EmployeeId &&
                                 t.Status == "Completed");

            var pendingLeavesCount = await _context.LeaveRequests
                .CountAsync(lr => lr.EmployeeEmail == employee.User.Email &&
                                  lr.Status == "Pending");

            var activeProjectsForProgress = await _context.Projects
                .Where(p => p.IsActive &&
                            (p.ManagerEmail == employee.User.Email ||
                             _context.Tasks.Any(t => t.ProjectId == p.ProjectId &&
                                                    t.AssignedTo == employee.EmployeeId)))
                .Select(p => new
                {
                    p.ProjectId,
                    p.Progress
                })
                .ToListAsync();

            double overallProgress = activeProjectsForProgress.Any()
                ? activeProjectsForProgress.Average(p => p.Progress ?? 0)
                : 0;

            ViewBag.FullName              = currentUser.FullName ?? currentUser.UserName ??
"User";     ViewBag.ActiveProjectsCount   = activeProjectsCount;
            ViewBag.MyTasksCount          = myTasksCount;
            ViewBag.MyCompletedTasksCount = myCompletedTasksCount;
            ViewBag.PendingLeavesCount    = pendingLeavesCount;
            ViewBag.OverallProgress       = Math.Round(overallProgress, 0);


            return View();
        }
    }
}
```

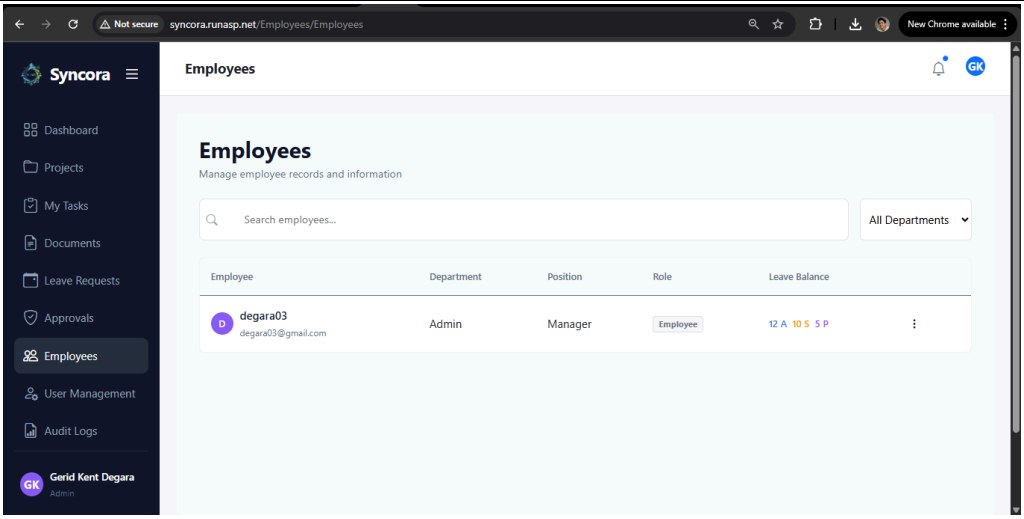
Update/Edit Page (Screenshot)	 This image shows an "Edit Employee" pop-up window within an employee module. Its main function is to allow administrators to update and manage specific details for an individual worker, such as their department, job role, and phone number. Beyond basic contact information, the screen is used to track and adjust leave balances, including annual, sick, and personal days. Once the necessary changes are made, the user can either save the new data or cancel the action to keep the original records.

Update/Edit Page - Source Code (Screenshot)	

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using IT15_SOWCS.Data;
using IT15_SOWCS.Models;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace IT15_SOWCS.Controllers
{
    [Authorize(Roles = "Admin")]
    public class EmployeesController : Controller
    {
        private readonly ApplicationDbContext _context;

        public EmployeesController(ApplicationDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public async Task<IActionResult> Index()
        {
            var employees = await _context.Employees
                .Include(e => e.User)
                .AsNoTracking()
                .Select(e => new
                {
                    e.EmployeeId,
                    Email = e.User != null ? e.User.Email : null,
                    FullName = e.User != null ? e.User.FullName : null,
                    UserName = e.User != null ? e.User.UserName : null,
                    e.Department,
                    e.Position,
                    e.Role,
                    e.ContactNumber,
                    e.AnnualLeaveBalance,
                    e.SickLeaveBalance,
                    e.PersonalLeaveBalance,
                    AvatarInitial = !string.IsNullOrEmpty(e.User?.FullName)
                        ? e.User.FullName.Substring(0, 1).ToUpper()
                        : (!string.IsNullOrEmpty(e.User?.UserName)
                            ? e.User.UserName.Substring(0, 1).ToUpper()
                            : "?")
                })
                .OrderBy(e => e.FullName ?? e.UserName ?? "")
                .ToListAsync();

            return View(employees);
        }

        [HttpGet]
        public async Task<IActionResult> EditPartial(string id)
        {
            if (string.IsNullOrWhiteSpace(id))
            {
                return BadRequest("Employee ID is required.");
            }

            var employee = await _context.Employees
                .Include(e => e.User)
                .FirstOrDefaultAsync(e => e.EmployeeId == id || e.User.Email == id);

            if (employee == null)
            {
                return NotFound("Employee not found.");
            }

            return PartialView("_EditEmployeePartial", employee);
        }
```

```
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> UpdateEmployee([FromBody] EmployeeUpdateDto
model)  {
            if (!ModelState.IsValid)
            {
                return Json(new { success = false, message = "Invalid data submitted." });
            }

            var employee = await _context.Employees
                .FirstOrDefaultAsync(e => e.EmployeeId == model.EmployeeId);

            if (employee == null)
            {
                return Json(new { success = false, message = "Employee not found." });
            }

            employee.Department            = model.Department;
            employee.Position              = model.Position;
            employee.Role                  = model.Role;
            employee.ContactNumber         = model.ContactNumber;

            employee.AnnualLeaveBalance    = model.AnnualLeaveBalance;
            employee.SickLeaveBalance      = model.SickLeaveBalance;
            employee.PersonalLeaveBalance  = model.PersonalLeaveBalance;

            try
            {
                _context.Employees.Update(employee);
                await _context.SaveChangesAsync();

                return Json(new
                {
                    success = true,
                    message = "Employee updated successfully."
                });
            }
            catch (DbUpdateException ex)
            {
                return Json(new
                {
                    success = false,
                    message = "Failed to save changes. Database error occurred."
                });
            }
            catch (Exception)
            {
                return Json(new
                {
                    success = false,
                    message = "An unexpected error occurred."
                });
            }
        }
    }
```

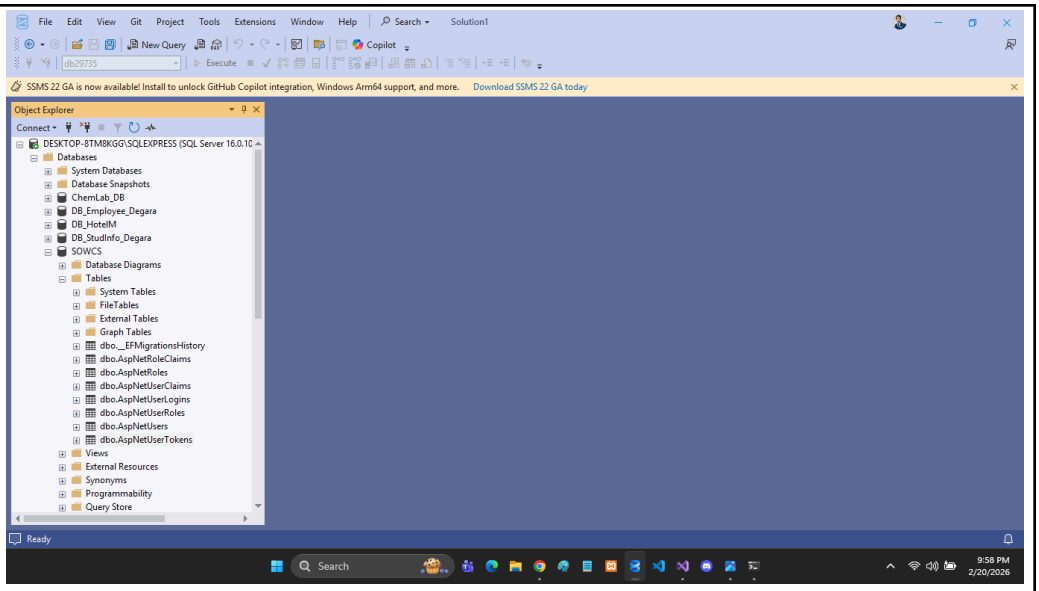
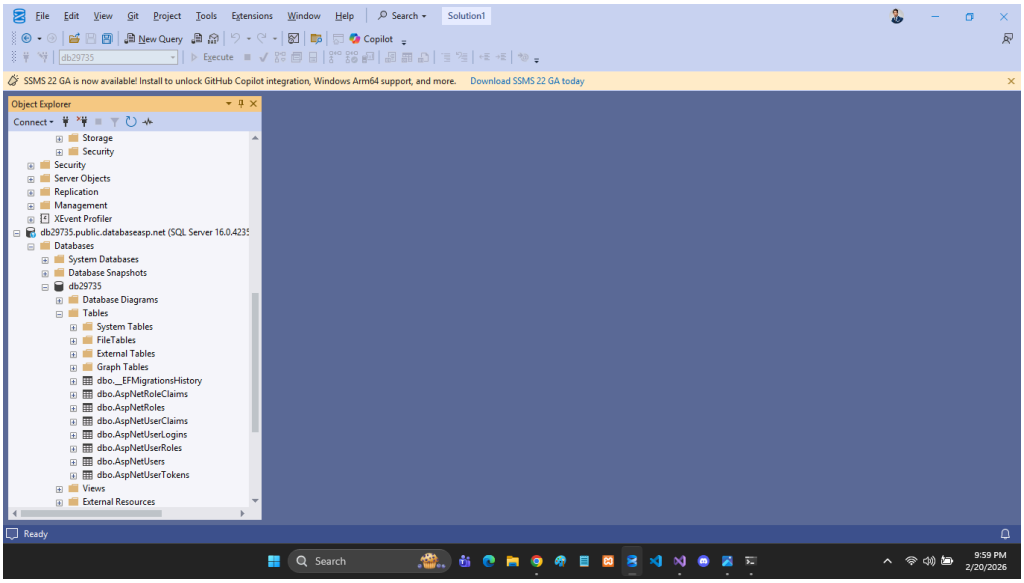
Archive (Screenshot)	**Image and Description Function (Whole Browser Page)**
Archive – Source Code (Screenshot)	**Source Code from C#_ASP. NET Core**

Display Records (Screenshot)	 This image shows the Employees page, where an HR officer can manage and view staff records. It features a table listing employee details such as their department, position, role, and current leave balance. HR officers can also use the search bar or department filter to quickly find specific team members.

Display Records - Source Code (Screenshot)	

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using IT15_SOWCS.Data;
using IT15_SOWCS.Models;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace IT15_SOWCS.Controllers
{
    [Authorize(Roles = "Admin")]
    public class EmployeesController : Controller
    {
        private readonly ApplicationDbContext _context;

        public EmployeesController(ApplicationDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public async Task<IActionResult> Index()
        {
            var employees = await _context.Employees
                .Include(e => e.User)
                .AsNoTracking()
                .Select(e => new
                {
                    e.EmployeeId,
                    Email = e.User != null ? e.User.Email : null,
                    FullName = e.User != null ? e.User.FullName : null,
                    UserName = e.User != null ? e.User.UserName : null,
                    e.Department,
                    e.Position,
                    e.Role,
                    e.ContactNumber,
                    e.AnnualLeaveBalance,
                    e.SickLeaveBalance,
                    e.PersonalLeaveBalance,
                    AvatarInitial = !string.IsNullOrEmpty(e.User?.FullName)
                        ? e.User.FullName.Substring(0, 1).ToUpper()
                        : (!string.IsNullOrEmpty(e.User?.UserName)
                            ? e.User.UserName.Substring(0, 1).ToUpper()
                            : "?")
                })
                .OrderBy(e => e.FullName ?? e.UserName ?? "")
                .ToListAsync();

            return View(employees);
        }

        [HttpGet]
        public async Task<IActionResult> EditPartial(string id)
        {
            if (string.IsNullOrWhiteSpace(id))
            {
                return BadRequest("Employee ID is required.");
            }

            var employee = await _context.Employees
                .Include(e => e.User)
                .FirstOrDefaultAsync(e => e.EmployeeId == id || e.User.Email == id);

            if (employee == null)
            {
                return NotFound("Employee not found.");
            }

            return PartialView("_EditEmployeePartial", employee);
        }
```

```
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> UpdateEmployee([FromBody] EmployeeUpdateDto
model)  {
            if (!ModelState.IsValid)
            {
                return Json(new { success = false, message = "Invalid data submitted." });
            }

            var employee = await _context.Employees
                .FirstOrDefaultAsync(e => e.EmployeeId == model.EmployeeId);

            if (employee == null)
            {
                return Json(new { success = false, message = "Employee not found." });
            }

            employee.Department            = model.Department;
            employee.Position              = model.Position;
            employee.Role                  = model.Role;
            employee.ContactNumber         = model.ContactNumber;

            employee.AnnualLeaveBalance    = model.AnnualLeaveBalance;
            employee.SickLeaveBalance      = model.SickLeaveBalance;
            employee.PersonalLeaveBalance  = model.PersonalLeaveBalance;

            try
            {
                _context.Employees.Update(employee);
                await _context.SaveChangesAsync();

                return Json(new
                {
                    success = true,
                    message = "Employee updated successfully."
                });
            }
            catch (DbUpdateException ex)
            {
                return Json(new
                {
                    success = false,
                    message = "Failed to save changes. Database error occurred."
                });
            }
            catch (Exception)
            {
                return Json(new
                {
                    success = false,
                    message = "An unexpected error occurred."
                });
            }
        }
    }
```

Database – SQL Server (Local) Screenshot	
Database – Deployed/Online Screenshot	
Date Submitted:	**February 20, 2026**
Teacher's Feedback	_____ _____ _____ _____

	_____ _____ _____ _____ Student's Signature (after feedbacking) Teacher's Signature